1 Re-evaluating Prime Factorization (PFZ) Leakage in z/z' Encodings

While not an entirely novel concept, a re-evaluation of the previously discarded "DFT + z/z'" encoding reveals that it may not be as insecure as initially assumed[cite: 1]. Previously, it was believed that this structure inherently leaks the Prime Factorization (PFZ) of the modulus $N = pq$. However, this leakage can be prevented by ensuring symmetry across the moduli p and q[cite: 1].

If the moduli p and q are treated separately, the structures appear as follows:

p		q	
0	0	0	0
a	b	c	d
w	x	y	z

Instead, by treating $N = pq$ holistically as a product (where a is the message and $a < \min(p, q)$), we establish a uniform structure[cite: 1]:

$N = pq$	
0	0
a	r
r	c

$\Longrightarrow$

p		q	
0	0	0	0
a	r	a	R
r	c	R	C

By utilizing the exact same values for a and c across both p and q, we ensure that if a zero occurs in one modulus, it simultaneously occurs in the other[cite: 1]. Consequently, attempting to compute an inverse via Gaussian elimination forces an explicit division by zero globally, rendering the operation impossible without leaking the PFZ[cite: 1].

1.1 The Zero-Test Vulnerability

While intentional zeros in a single factor are avoided, a plaintext zero ($a = 0$) still presents a challenge. The inability to perform Gaussian elimination functionally acts as a "zero-test"[cite: 1]. An adversary could brute-force the plaintext by continuously checking if $(x - c^2) == 0$ evaluates to true[cite: 1].

Mitigating this vulnerability must be handled at the application layer:

- **Value Scaling:** Applications must operate on sufficiently large numbers, rendering brute-force attacks computationally infeasible[cite: 1].
- **Dummy Values:** The introduction of decoy values ensures an adversary can never be certain whether a discovered zero belongs to a legitimate computation or a dummy variable[cite: 1].
- **Multiplicative Masking:** Registers should ideally maintain values that are consistently multiplied by masking factors, as typical application registers might otherwise contain easily guessable small integers[cite: 1].

2 Catastrophic Failure via Known Plaintext-Ciphertext Attacks

Despite the theoretical soundness of the PFZ protection, practical exploitation demonstrates that the scheme remains fundamentally broken[cite: 1]. Given a known Plaintext-Ciphertext (PT-CT) pair, an adversary can easily resolve the underlying System of Linear Equations (SLE), thereby breaking the z/z' encoding and compromising the entire encryption scheme[cite: 1].

Specifically, the z/z' encryption collapses if a zero exists within the plaintext. During the SLE resolution, this produces a complex number with a norm of zero[cite: 1]. Once such a null-element is exposed, the z/z' encoding is trivially solvable[cite: 1].

Neither Quaternions nor basic C^2 algebras mitigate this vulnerability, as they suffer from the exact same mathematical flaw: exposing a zero structurally destroys the encoding[cite: 1].

3 The Invertibility Problem in Algebraic Rings

A broader mathematical limitation is the inability to construct a field or ring that supports both addition and multiplication while simultaneously making the computation of a multiplicative inverse computationally hard[cite: 1]. Even within polynomial rings, inverses can be derived trivially using the Extended Euclidean Algorithm[cite: 1].

4 Proposed Solution: Non-Leaking C^2 Extensions

To circumvent the vulnerability where a lack of an inverse leaks the PFZ or the encoding, the concept of non-PFZ-leaking complex numbers must be extended to the C^2 space[cite: 1]. The goal is to construct a system where the norm of a C^2 number reliably evaluates to $(0, 0)$ whenever an inverse does not exist, thereby protecting both the z/z' encoding and the PFZ[cite: 1].

This leads to the following logical encoding structure:

$$[r_1, r_2, m, m] \longrightarrow C^2 \tag{1}$$

Under this mapping, the vector $[r_1, r_2, 0, 0]$ will always yield a norm of $(0, 0)$[cite: 1]. Furthermore, it must be mathematically impossible for the two m components to diverge in value[cite: 1].

4.1 Derivation of C^2 Basis Vectors

Let $x = (p + 1)/2$ and $y = \sqrt{-x^2}$[cite: 1]. The formulas for deriving the C^2 basis vectors are as follows:

For the first base structure:

$$(x, 0, y, 0) \longrightarrow (x, y) \longrightarrow (x^2, xy, xy, y^2) \tag{2}$$

$$\longrightarrow (x, -y) \longrightarrow (x^2, -xy, xy, -y^2) \tag{3}$$

For the conjugate structure:

$$(x, 0, -y, 0) \longrightarrow (x, y) \longrightarrow (x^2, xy, -xy, -y^2) \tag{4}$$

$$\longrightarrow (x, -y) \longrightarrow (x^2, -xy, -xy, y^2) \tag{5}$$

By combining specific vectors (e.g., b_0 with b_1, or b_2 with b_3), the original structures $(x, 0, y, 0)$ or $(x, 0, -y, 0)$ are recovered, establishing a secure baseline for the encryption mapping[cite: 1].